

p0528R2

The Curious Case of Padding Bits, Featuring Atomic Compare-and-Exchange

Published Proposal, 16 March 2018

This version:

<http://wg21.link/P0528r2>

Authors:

[JF Bastien](#) (Apple)

[Michael Spencer](#) (Sony Playstation)

Audience:

CWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Source:

github.com/jfbastien/papers/blob/master/source/P0528r2.bs

Abstract

Compare-and-exchange on a struct with padding bits should Just Work.

Table of Contents

1 Edit History

1.1 r1 → r2

1.2 r0 → r1

2 Proposed Wording

References

Informative References

This issue has been discussed by the authors at every recent Standards meetings, yet a full solution has been elusive despite helpful proposals. We believe that this proposal can fix this oft-encountered problem once and for all.

[\[P0528r0\]](#) details extensive background on this problem (not repeated here), and proposed standardizing a trait, `has_padding_bits`, and using it on `compare_and_exchange_*`. [\[P0528r1\]](#) applied EWG guidance and simply added wording directing implementations to ensure that the desired behavior occur. At SG1's request this paper follows EWG's guidance but uses different wording.

§ 1. Edit History

§ 1.1. r1 → r2

In Jacksonville, SG1 supported the paper but suggested an alternate way to approach the wording than the one EWG proposed in Albuquerque: don't talk about contents of the memory, but rather discuss the value representation to describe compare-and-exchange. This paper follows SG1's guidance and offers different wording, with the intent that the semantics be equivalent. EWG reviewed the updated wording and voted to support it and forward to Core.

§ 1.2. r0 → r1

In Albuquerque, EWG voted to make the padding bits of `atomic` and the incoming value of `T` have a consistent value for the purposes of read/modify/write atomic operations?

Purposefully not addressed in this paper:

- `union` with padding bits
- Types with trap representations

§ 2. Proposed Wording

Edit §17 and onwards as follows:

Requires: The `failure` argument shall not be `memory_order::release` nor `memory_order::acq_rel`.

Effects: Retrieves the value in `expected`. It then atomically compares the ~~contents of the memory pointed to by this~~ `value representation of *this` for equality with that previously retrieved from `expected`, and if true, replaces the ~~contents of the memory pointed to by this~~ `value representation of *this` with that in `desired`. If and only if the comparison is true, memory is affected according to the value of `success`, and if the comparison is false, memory is affected according to the value of `failure`. When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`. If and only if the comparison is false then, after the atomic operation, the ~~contents of the memory~~ `the value representation` in `expected` are replaced by the value `representation` read from the memory pointed to by `this` during the atomic comparison. If the operation returns `true`, these operations are atomic read-modify-write operations on the memory pointed to by `this`. Otherwise, these operations are atomic load operations on that memory.

Returns: The result of the comparison.

[*Note:*

For example, the effect of `compare_exchange_strong` on objects without padding bits is

```
if (memcmp(this, &expected, sizeof(*this)) == 0)
    memcpy(this, &desired, sizeof(*this));
else
    memcpy(expected, this, sizeof(*this));
```

—end note]

[*Example:*

The expected use of the compare-and-exchange operations is as follows. The compare-and-exchange operations will update `expected` when another iteration of the loop is needed.

```
expected = current.load();
do {
    desired = function(expected);
} while (!current.compare_exchange_weak(expected, desired));
```

—end example]

[*Example:*

Because the expected value is updated only on failure, code releasing the memory containing the `expected` value on success will work. E.g. list head insertion will act atomically and would not introduce a data race in the following code:

```
do {
    p->next = head; // make new list node point to the current head
} while (!head.compare_exchange_weak(p->next, p)); // try to insert
```

—end example]

Implementations should ensure that weak compare-and-exchange operations do not consistently return `false` unless either the atomic object has value different from `expected` or there are concurrent modifications to the atomic object.

Remarks: A weak compare-and-exchange operation may fail spuriously. That is, even when the contents of memory referred to by `expected` and `this` are equal, it may return `false` and store back to `expected` the same memory contents that were originally there.

[Note:

This spurious failure enables implementation of compare-and-exchange on a broader class of machines, e.g., load-locked store-conditional machines. A consequence of spurious failure is that nearly all uses of weak compare-and-exchange will be in a loop. When a compare-and-exchange is in a loop, the weak version will yield better performance on some platforms. When a weak compare-and-exchange would require a loop and a strong one would not, the strong one is preferable.

—end note]

[Note:

The `memcpy` and `memcmp` semantics of the compare-and-exchange operations may result in failed comparisons for values that compare equal with `operator==` if the underlying type has padding bits which sometimes participate in the object's representation, trap bits, or alternate representations of the same value other than those caused by padding bits which never participate in the object's representation. Notably, on implementations conforming to ISO/IEC/IEEE 60559, floating-point `-0.0` and `+0.0` will not compare equal with `memcmp` but will compare equal with `operator==`, and NaNs with the same payload will compare equal with `memcmp` but will not compare equal with `operator==`.

—end note]

[Note:

Compare-and-exchange acts on an object's value representation, ensuring that padding bits which never participate in the object's representation are ignored.

As a consequence, the following code is guaranteed to avoid spurious failure:

```
struct padded {
    char clank = 0x42;
    // Padding here.
    unsigned biff = 0xC0DEFEFE;
};
atomic<padded> pad = ATOMIC_VAR_INIT({});

bool zap() {
    padded expected, desired { 0, 0 };
    return pad.compare_exchange_strong(expected, desired);
}
```

—end note]

[Note:

Types which contain bits that sometimes participate in the object's representation, such as a `union` containing a type with padding bits and a type without, may always fail compare-and-exchange when these bits are not participating in the object's representation because they have an indeterminate value.

—end note

§ References

§ Informative References

[P0528r0]

JF Bastien, Michael Spencer. [The Curious Case of Padding Bits, Featuring Atomic Compare-and-Exchange](https://wg21.link/p0528r0). 12 November 2016. URL: <https://wg21.link/p0528r0>

[P0528r1]

JF Bastien, Michael Spencer. [The Curious Case of Padding Bits, Featuring Atomic Compare-and-Exchange](https://wg21.link/p0528r1). URL: <https://wg21.link/p0528r1>